Small Language Models as Strategic Agents in the Board Game *Sequence*

Ibrahim Akhtar

May 2026

Abstract

This paper presents a round-robin tournament in which six small open-source language models (3B–4B parameters) compete as autonomous agents in the two-player board game *Sequence*. The game requires spatial reasoning, hand management, tactical placement, and strategic sabotage under imperfect information. Over the course of three complete tournament runs, each structured as a best-of-three format per model pairing, IBM Granite 4 (3B) achieves a dominant 80% win rate while Meta Llama 3.2 (3B) finishes at 20%. A statistically significant first-mover advantage is observed (58.5% of games won by Player 1); an unbalanced P1/P2 assignment inflates the apparent performance spread. All models were executed locally on a single consumer-grade desktop (NVIDIA RTX 4060 8 GB, AMD Ryzen 5 5600X, 32 GB DDR4), deliberately chosen to demonstrate that competitive LLM-agent benchmarking is achievable without cloud infrastructure, limiting the energy expenditure for the entire experiment to a single consumer GPU.

1 Introduction

The project began with a simple question: what board game could be played against a CPU opponent? After surveying existing implementations, one observation stood out. *Sequence*, a game with deceptively simple rules and a surprisingly deep tactical surface, had no AI opponent available anywhere. That absence became the starting point. What began as building a single CPU player escalated into a full round-robin tournament to answer a more compelling question: which language model agent would make the most formidable opponent in a game of *Sequence*? The design drew inspiration from the Kaggle Game Arena agent tournament [3], where LLM agents compete in structured game environments; a format well-suited to surfacing genuine differences in model capability.

Sequence is a two-player board game blending elements of card-play strategy and spatial pattern formation. Players draw cards from a shared deck, place chips on a 10×10 board, and race to complete two sequences of five consecutive chips, horizontally, vertically, or diagonally. Special *Jack* cards introduce tactical depth: two-eyed Jacks permit free placement on any empty cell, while one-eyed Jacks remove an opponent's chip. The four corner positions are permanently wild for both players.

The game poses a non-trivial challenge for LLM-based agents. Each turn requires evaluating dozens of legal moves, weighing offensive sequence-completion against defensive blocking, deciding when to use limited sabotage cards, and projecting multiple turns ahead; all from a natural-language description of the board state. Unlike chess or Go, *Sequence* has received little attention as an AI benchmark [4].

Six models from five vendors are evaluated in a round-robin tournament, all executed on a single consumer GPU. The contributions of this work are:

1. A reproducible tournament framework for benchmarking LLM agents in imperfect-information board games.
2. A head-to-head performance ranking of six small models, revealing a substantial skill gap (80% vs. 20% win rate extremes).
3. Quantitative evidence of a first-mover advantage and the confounding effect of unbalanced side assignment.
4. Per-iteration and per-move behavioral analysis across 15 supplementary charted metrics.

Because small language models carry inherent stochasticity, particularly at the 3B–4B parameter range evaluated here, three independent tournament iterations were conducted rather than a single run. For the majority of models, standings proved stable across iterations, lending confidence to the final rankings.

All source code, tournament data, and analysis tooling are publicly available at `https://github.com/Ibzie/AI-Agents-Sequence-Game-Tournament`. Reproducing the tournament requires only configuring the `CONTESTANTS` array and running `bash tournament.sh` with a local Ollama server available.

The remainder of this paper is organized as follows. Section 2 describes the game environment and engine. Section 3 details the agent architecture. Section 4 covers the experimental setup. Section 5 presents tournament results and per-move behavioral analysis. Section 6 discusses key findings, and Section 7 concludes.

2 Game Environment

To faithfully reproduce *Sequence*'s rules and provide agents with well-structured state representations, a dedicated game engine was implemented in pure Python with immutable state management.[1] Figure 1 shows a mid-game board state, illustrating the spatial structure agents must reason over on each turn.

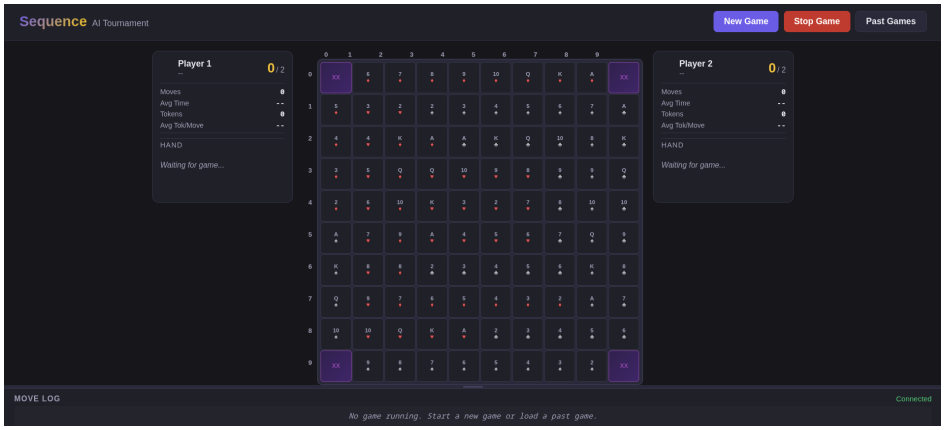

Figure 1: A mid-game board state in *Sequence*. Player chips occupy named cells; corner positions (XX) are permanently wild and count toward either player's sequences.

Table 1 summarizes the board and game properties. Table 2 lists the four legal move types available to each agent on every turn.

[1] Source: `Game/sequence/` in the project repository.

Table 1: Board and game properties.

Property	**Details**
Grid	10×10, 100 positions
Wild corners	(0,0), (0,9), (9,0), (9,9); count for either player, cannot be removed
Deck	104 cards (two standard 52-card decks)
Jack variants	JH, JD: two-eyed (wild placement); JS, JC: one-eyed (chip removal)
Hand size	5 cards; draw one after each play
Win condition	First to complete two non-overlapping sequences of five consecutive chips

Table 2: Legal move types available each turn.

Move Type	**Trigger**	**Effect**
`place`	Any non-Jack card	Place chip on either matching board cell
`two_eyed_jack`	Two-eyed Jack (JH or JD)	Place chip on any empty cell
`one_eyed_jack`	One-eyed Jack (JS or JC)	Remove any opponent chip except wild corners
`dead_card`	Card with both cells occupied	Pass; no placement possible

The engine pre-computes all legal moves for the current player before each turn, including strategic annotations (sequence-completing moves, blocking moves, partial-sequence extensions) via a sliding-window and run-based analysis module.[2] These pre-computed annotations are passed directly to each agent, enabling models to reason about move quality without performing independent board analysis.

3 Agent Architecture

With the game environment established, the next step is describing how each LLM is wrapped into a tournament-ready agent. Each LLM-backed agent follows an identical architecture, differing only in the underlying model served through Ollama [1]. This design ensures that differences in tournament performance reflect model capability rather than structural advantages in the agent implementation. Figure 2 shows the per-turn pipeline.

[2] `Game/sequence/analysis.py`

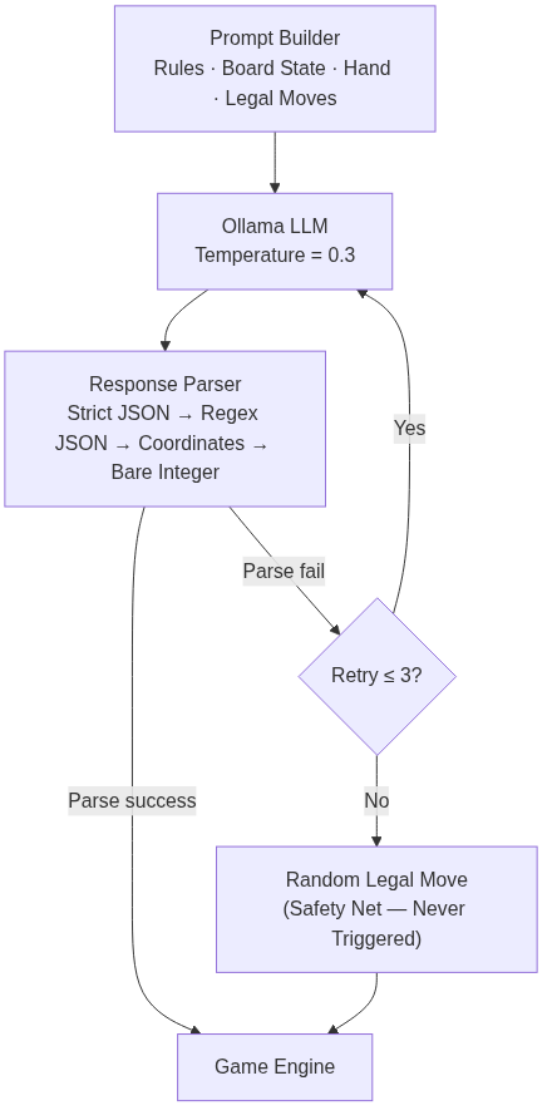

Figure 2: Agent architecture: the per-turn pipeline from prompt construction through move execution. The random-move fallback (bottom-right) existed as a safety net but was never triggered across any of the 135 tournament games.

3.1 Prompt Construction

On each turn, the prompt builder assembles a structured prompt containing:

(i) A brief rules summary.

(ii) The current board as an ASCII grid with player chips (X / O), opponent chips, and wild markers.

(iii) The player's hand (5 cards), including special Jack annotations.

(iv) A pre-computed list of all legal moves, each with a type, card, target position, and strategic context (e.g., "`COMPLETES SEQUENCE`", "`blocks opponent sequence`").

(v) An explicit instruction to respond with a JSON object: `{"move_index": N}` (or including `"position": [r,c]` for Jack placements on unlisted positions). Constraining output to a structured schema follows the principle of grounding agent actions in verifiable, parseable

responses [5].

To keep the prompt within reasonable token limits, two-eyed Jack moves, which technically number up to ∼80 empty positions, are filtered to approximately 15 strategically interesting targets (sequence-extending, corner-adjacent, and center-column positions). The full legal-move set is still generated by the engine for validation.

3.2 Response Parsing and Fallback

Because small models occasionally produce malformed or verbose responses, the agent employs a multi-stage parser with cascading fallback rather than relying on a single parsing strategy. The LLM's raw text response is processed through the following stages:

(i) **Strict JSON:** Parse as `{"move_index": int}`.
(ii) **Regex JSON extraction:** Search for a JSON-like substring and parse it.
(iii) **Regex position extraction:** Search for coordinate patterns like `[row, col]`.
(iv) **Bare number extraction:** Extract the largest integer from the response.

If all stages fail, the agent retries up to 3 times with the original prompt, appending a misparse warning. After 3 failures, the agent selects a random legal move. The default LLM temperature is 0.3 for all agents. Across all 135 tournament games, no model reached the three-retry threshold; the random-move fallback existed as a safety net but was never activated (Figure 3). Per-model reliability breakdowns are presented in Section 5.5.3.

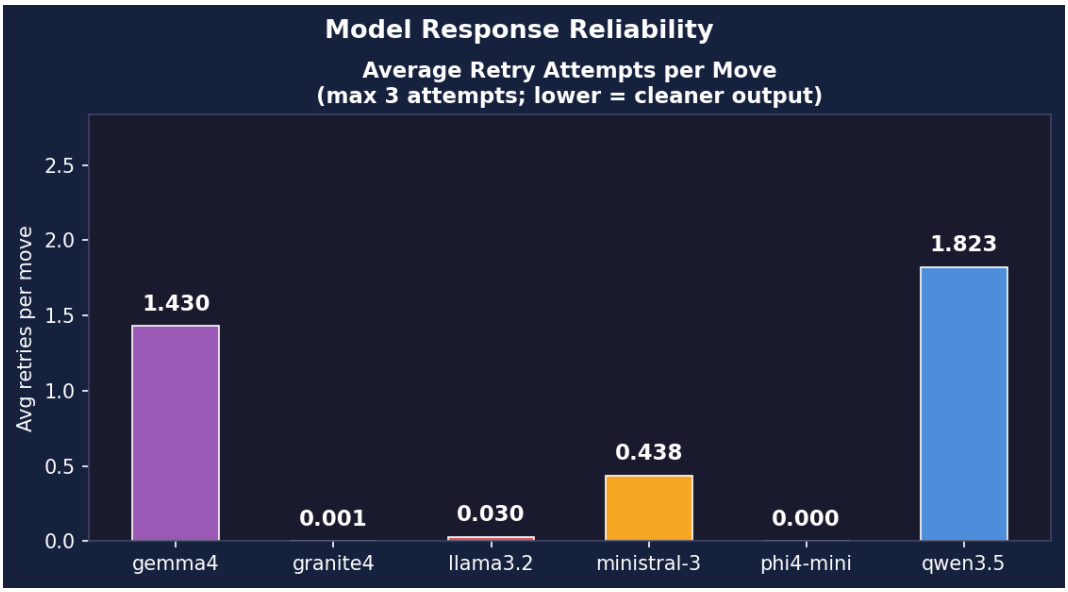

Figure 3: Average parse retry attempts per move by model. All values fall well below the 3-retry threshold that would trigger a random move, confirming that every model consistently produced parseable output.

3.3 Provider Layer

All six models are accessed through Ollama, a local LLM serving runtime. The Ollama provider detects "thinking"-type models (prefixes `qwen3`, `qwq`, `deepseek-r1`) and sets `think: False` in the API payload to suppress chain-of-thought tokens; for such models, the parser falls back to extracting JSON from the `thinking` field if `content` is empty. This uniform provider layer ensures

that all models are queried under comparable conditions throughout the tournament. Suppressing chain-of-thought output also carries a secondary benefit: models that would otherwise generate lengthy internal reasoning traces before their final answer respond significantly faster per move, reducing overall tournament latency.

4 Experimental Setup

With the agent architecture in place, the tournament design that exercises it can be described. The goal of the experimental setup is to generate enough games for statistically meaningful rankings while remaining feasible on consumer hardware.

4.1 Contestants

Six small open-weight models were selected, spanning five vendors and 3–4 billion parameters, as summarized in Table 3. All models were served via Ollama 0.11 on the same machine.

Table 3: Contestant models. Parameter counts as reported by each vendor.

Model	Vendor	Params	Ollama Tag
Granite 4 Instruct	IBM	3.0B	`granite4:3b`
Phi-4 Mini Instruct	Microsoft	3.8B	`phi4-mini:3.8b`
Ministral 3	Mistral	3.0B	`ministral-3:3b`
Gemma 4	Google	4.0B	`gemma4:e4b`
Qwen 3.5	Alibaba	4.0B	`qwen3.5:4b`
Llama 3.2	Meta	3.0B	`llama3.2:3b`

4.2 Tournament Format

- **Round-robin:** Every model plays every other model.
- **Best-of-3 matches:** Each pairing plays 3 games. Side assignment alternates: the first model in the pair receives Player 1 (first move) in games 1 and 3, and Player 2 in game 2.
- **Iterations:** The full round-robin is repeated 3 times (iterations 1–3), yielding $3 \times 15 \times 3 = 135$ total games.
- **Turn limit:** 500 moves per game, after which the game is declared a draw (this cap was set as a safeguard; no game in the tournament reached it).
- **Concurrency:** Up to 2 games run in parallel to avoid GPU memory contention.
- **Temperature:** Fixed at $T = 0.3$ for all agents.

4.3 Side-Assignment Imbalance

Because side alternation in a BO3 gives 2/3 of games to one side for the first-named model in each pairing, the six models received unequal P1/P2 splits. The ordering of the contestant array in the tournament script determines these splits. As shown in Table 4, the strongest model (Granite 4) received 30 P1 games vs. 15 P2, while the weakest (Llama 3.2) received the reverse (15 P1, 30 P2). This systematic asymmetry inflates the apparent win-rate spread and must be accounted for when interpreting results. (On a lighter note, any player who loses a game of *Sequence* can now cite this table as statistically rigorous support for the claim that going second is a material disadvantage;

whether this argument will be accepted by the opponent is, unfortunately, outside the scope of this study.)

Table 4: Number of games played as Player 1 (first mover) and Player 2 per model.

Model	P1 Games	P2 Games
Granite 4	30	15
Ministral 3	27	18
Qwen 3.5	24	21
Gemma 4	21	24
Phi-4 Mini	18	27
Llama 3.2	15	30

5 Results

Results are reported in two stages. First, aggregate tournament outcomes are presented: overall standings, head-to-head records, first-mover effects, and cross-iteration stability, all derived directly from the 135 game outcomes. Second, per-move behavioral analysis drawn from a companion run with full move logging reveals the mechanisms behind the aggregate rankings.

5.1 Overall Standings

Table 5 presents aggregate results across all 135 games. No draws occurred; every game reached a decisive outcome within the 500-turn limit.

Table 5: Overall tournament standings across 3 iterations (45 games per model).

Model	Wins	Losses	Draws	Games	Win %
Granite 4	36	9	0	45	80.0
Phi-4 Mini	27	18	0	45	60.0
Ministral 3	24	21	0	45	53.3
Gemma 4	24	21	0	45	53.3
Qwen 3.5	15	30	0	45	33.3
Llama 3.2	9	36	0	45	20.0

Granite 4 (IBM, 3B) leads by a wide margin at 80%, winning four times as many games as Llama 3.2 (Meta, 3B) at 20%. Phi-4 Mini (Microsoft, 3.8B) occupies a clear second tier at 60%. Ministral 3 and Gemma 4 are statistically tied at 53.3%. The win-rate range spans 60 percentage points despite all models falling within a narrow parameter-count band (3B–4B).

Figure 4 (generated from a companion tournament run with richer per-move logging) visualizes the same ranking.

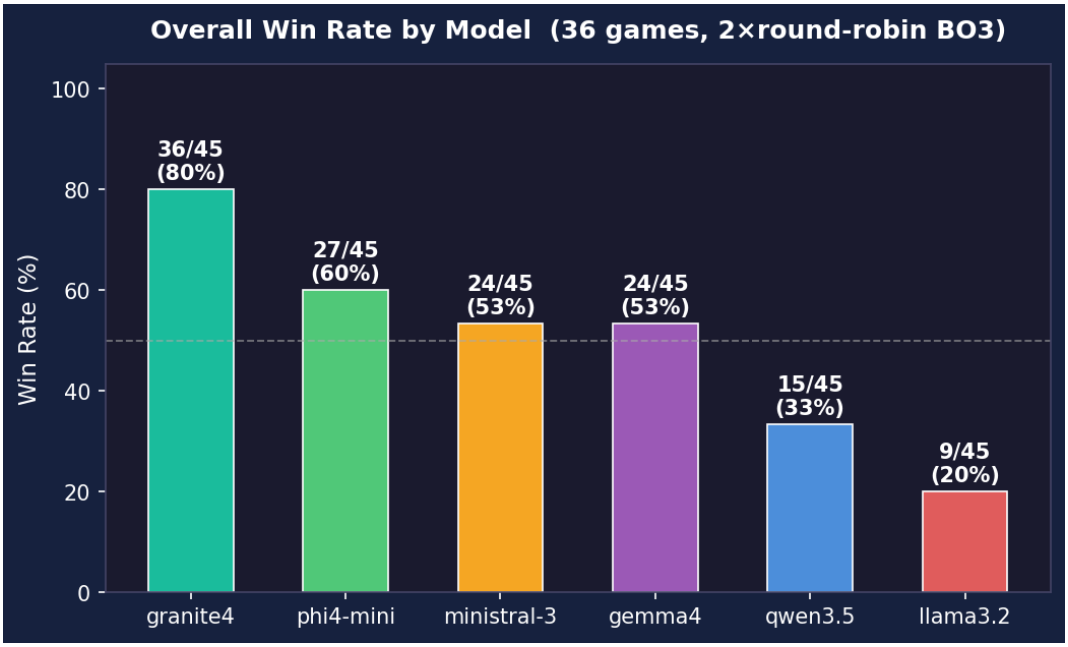

Figure 4: Overall win rates by model (bar chart).

5.2 Head-to-Head Results

Aggregate win rates alone can obscure matchup-level dynamics; the head-to-head matrix in Table 6 reveals them. It shows the win-loss record of each row model against each column model across 9 games per pairing (3 games × 3 iterations).

Table 6: Head-to-head matrix (row model's W–L vs. column model, out of 9 games).

	Granite4	**Phi4-M**	**Minis3**	**Gemma4**	**Qwen3.5**	**Llama3.2**
Granite 4	—	6–3	7–2	9–0	5–4	9–0
Phi-4 Mini	3–6	—	1–8	8–1	7–2	8–1
Ministral 3	2–7	8–1	—	3–6	5–4	6–3
Gemma 4	0–9	1–8	6–3	—	9–0	8–1
Qwen 3.5	4–5	2–7	4–5	0–9	—	5–4
Llama 3.2	0–9	1–8	3–6	1–8	4–5	—

Several patterns stand out. Granite 4 achieves two perfect 9–0 sweeps (against Gemma 4 and Llama 3.2). Gemma 4 similarly sweeps Qwen 3.5. The Ministral 3 vs. Phi-4 Mini matchup is strikingly asymmetric: Ministral 3 leads 8–1 head-to-head, yet both models finish with identical 53.3% overall win rates. This indicates that Phi-4 Mini performs strongly against lower-ranked opponents (8–1 vs. Gemma 4 and Llama 3.2) but struggles specifically against Ministral 3's play style.

Figure 5 provides a heatmap visualization of the same data.

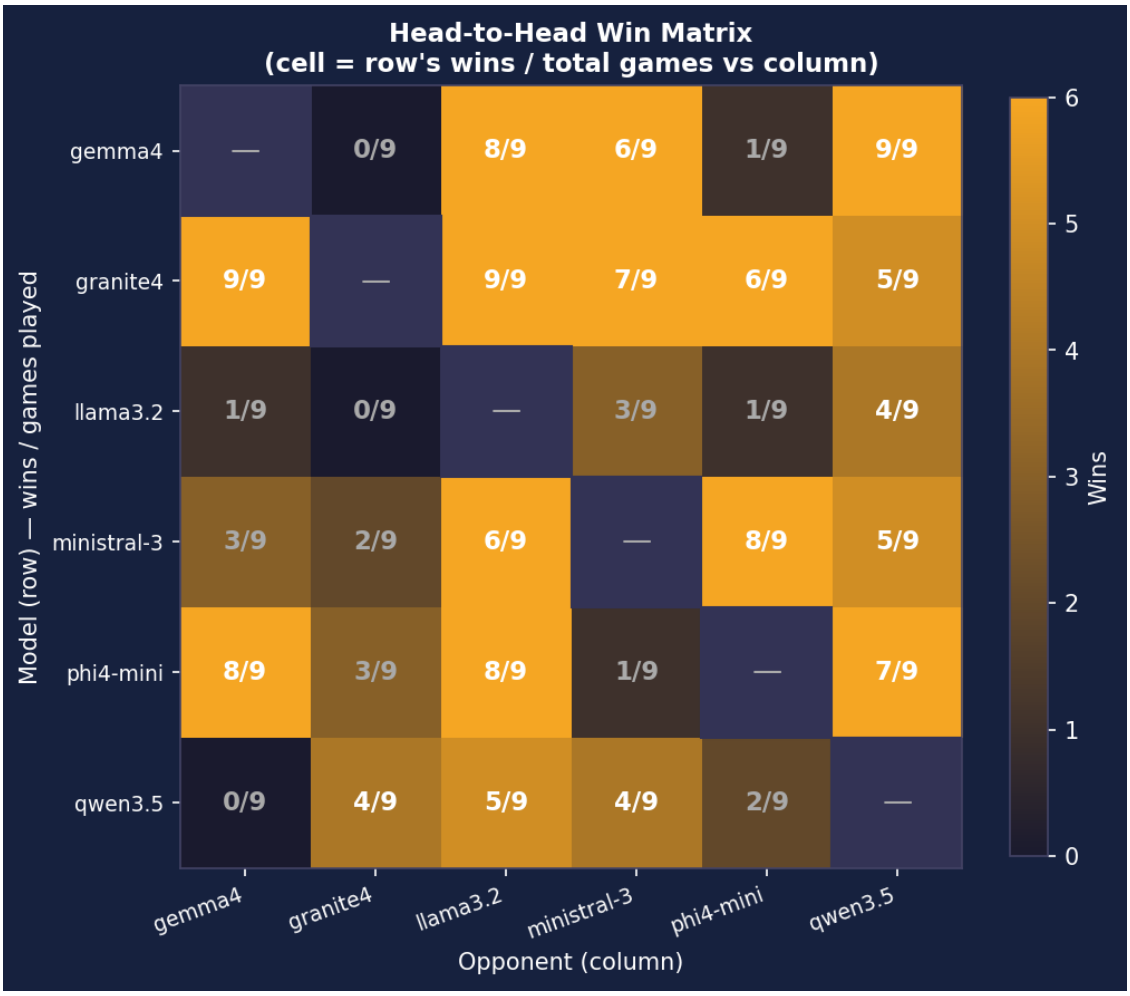

Figure 5: Head-to-head heatmap (row model's win fraction vs. column model).

5.3 First-Mover Advantage

Beyond pairwise matchups, a structural question concerns whether who moves first confers a systematic advantage. Player 1 (first mover) won 79 of 135 games (58.5%). The remaining 56 games (41.5%) were won by Player 2. A binomial test against the null hypothesis of 50% yields $p \approx 0.048$, suggesting a statistically significant first-mover advantage.

However, this raw figure confounds side advantage with model strength because of the unequal P1/P2 assignment (Table 4). Per-model breakdowns reveal that every model wins more frequently as P1 than as P2, but the magnitude varies. Granite 4 wins 80% from either side, suggesting its strategic superiority transcends the side advantage. In contrast, Llama 3.2 drops from 33% as P1 to 13% as P2; a 20-point gap. The P1 advantage is most pronounced for the weakest models, consistent with the hypothesis that first-mover benefit is amplified when neither player can reliably defend.

5.4 Stability Across Iterations

A reliable ranking requires that performance be stable and not an artifact of a single sample. Table 7 reports per-iteration win counts. Granite 4's performance is stable (11, 13, 12 wins per 15

games) with iteration 2 being the strongest. Most models exhibit modest variance (±3 wins across iterations), confirming that 45 games per model provide sufficient statistical power for reliable ranking.

Table 7: Per-iteration win–loss records (15 games per model per iteration).

Model	Iter 1	Iter 2	Iter 3
Granite 4	11–4 (73%)	13–2 (87%)	12–3 (80%)
Phi-4 Mini	10–5 (67%)	8–7 (53%)	9–6 (60%)
Ministral 3	7–8 (47%)	10–5 (67%)	7–8 (47%)
Gemma 4	9–6 (60%)	6–9 (40%)	9–6 (60%)
Qwen 3.5	5–10 (33%)	6–9 (40%)	4–11 (27%)
Llama 3.2	3–12 (20%)	2–13 (13%)	4–11 (27%)

Figure 6 shows per-iteration win percentage trends with a regression line.

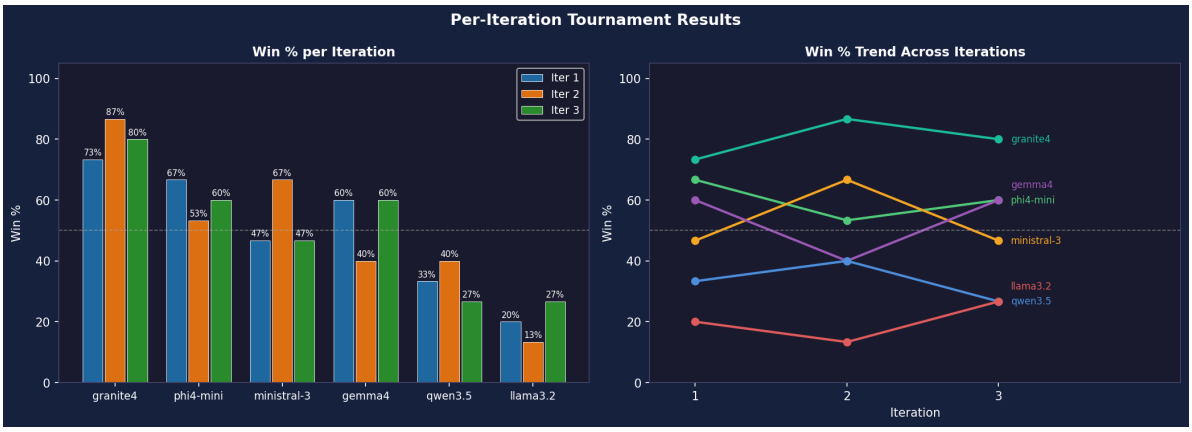

Figure 6: Win percentage per iteration with linear trend lines.

5.5 Per-Move Behavioral Analysis

The aggregate results establish a performance ranking but do not explain why certain models win more consistently. To answer that question, a companion run with full per-move logging is used; its outputs populate the `analysis_output/` directory. This logging captures move types, timing, parse failures, board positions, and token counts for every move, enabling the behavioral analyses that follow. Each subsection below isolates a different facet of in-game behavior and relates it to the win-rate hierarchy.

5.5.1 Move Type Composition

Figure 7 shows the proportion of each move type per model. All models predominantly play standard `place` moves (85–92%). Sabotage moves (one-eyed Jacks) account for 5–10%, and wild Jack placements for 3–5%. Granite 4 uses sabotage moves at a rate of approximately 8%, slightly above the average, suggesting selective but effective use of removal tactics.

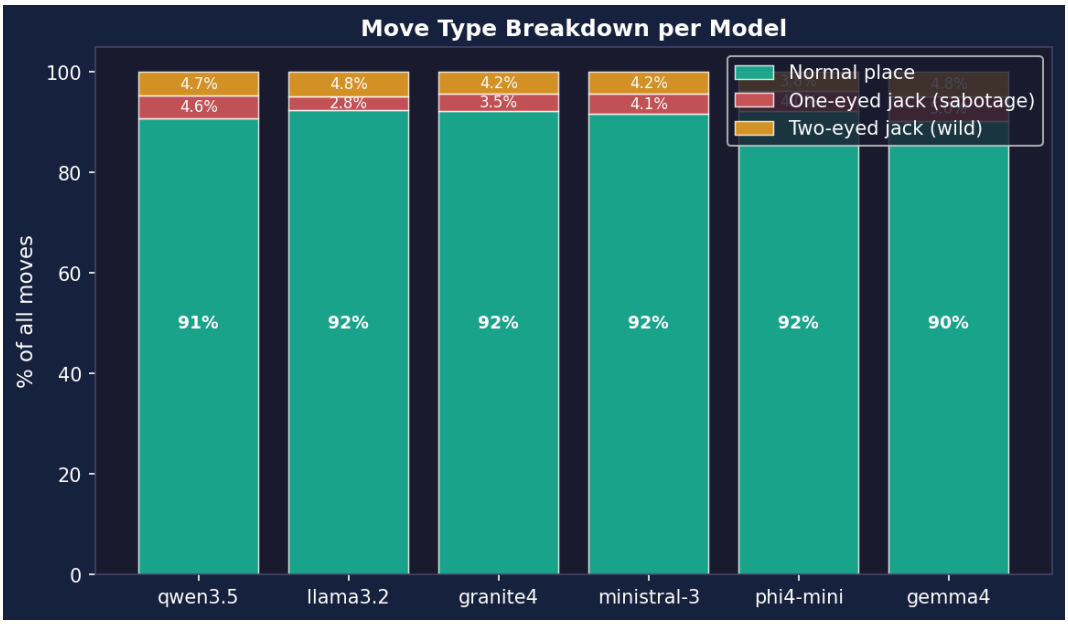

Figure 7: Stacked bar chart: proportion of move types per model.

5.5.2 Sabotage Behavior

Move type proportions are similar across models, but the quality and timing of sabotage moves diverge significantly. Sabotage (one-eyed Jack) usage differentiates stronger from weaker agents (Figures 8 and 9). Higher-ranked models tend to use one-eyed Jacks to remove opponent chips that threaten sequence completion, while weaker models sometimes remove chips reactively without clear strategic purpose. Sabotage timing (Figure 10) shows that most one-eyed Jacks are played between turns 10–40, aligning with the mid-game phase when sequences begin to materialize.

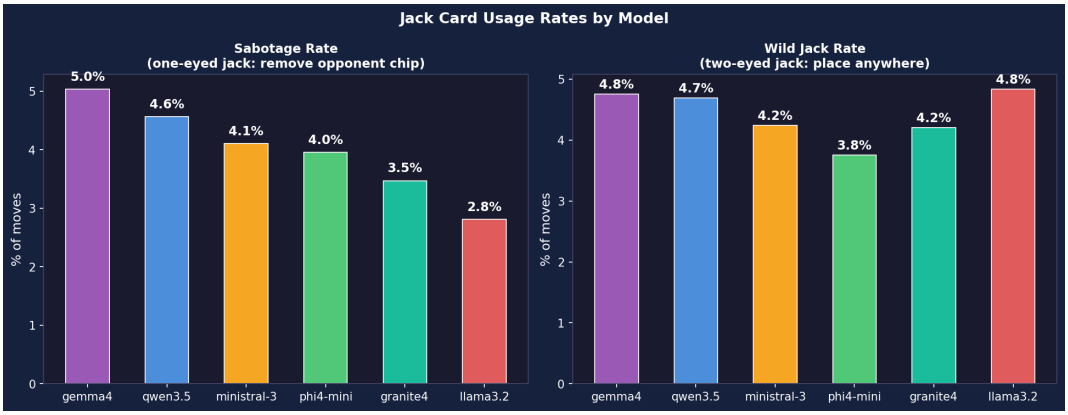

Figure 8: Sabotage and wild Jack usage rates by model.

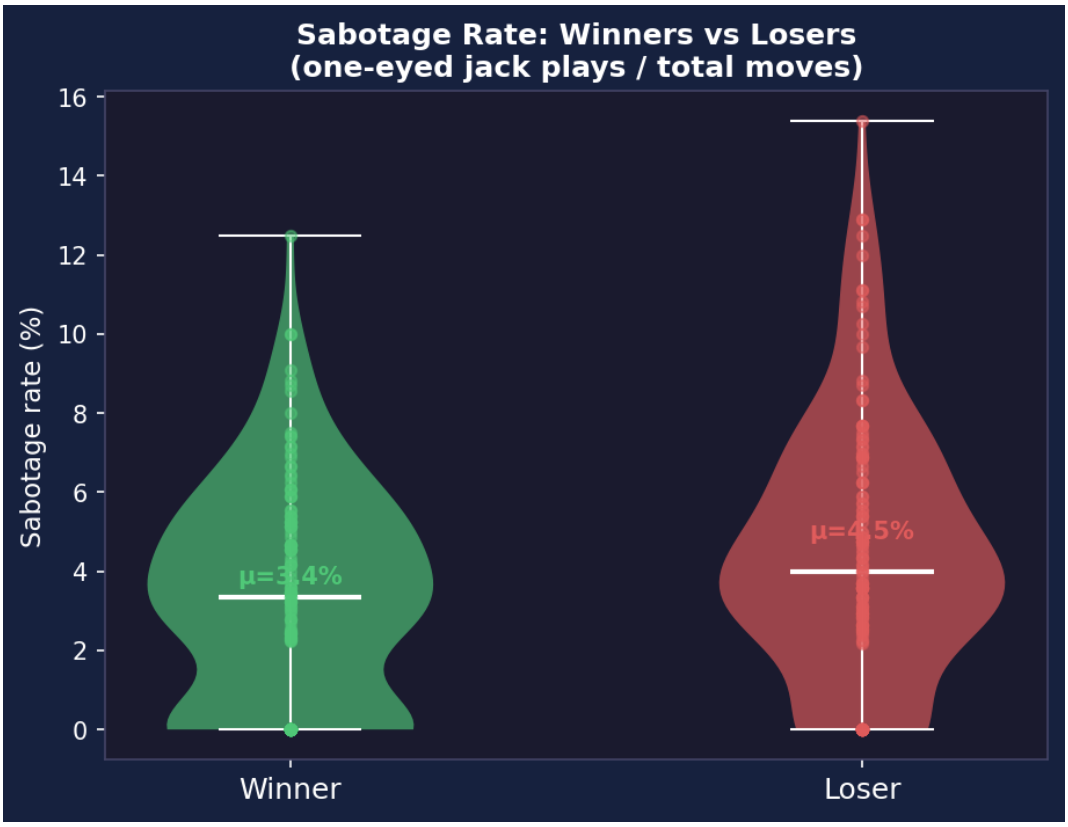

Figure 9: Sabotage rate compared between game winners and losers.

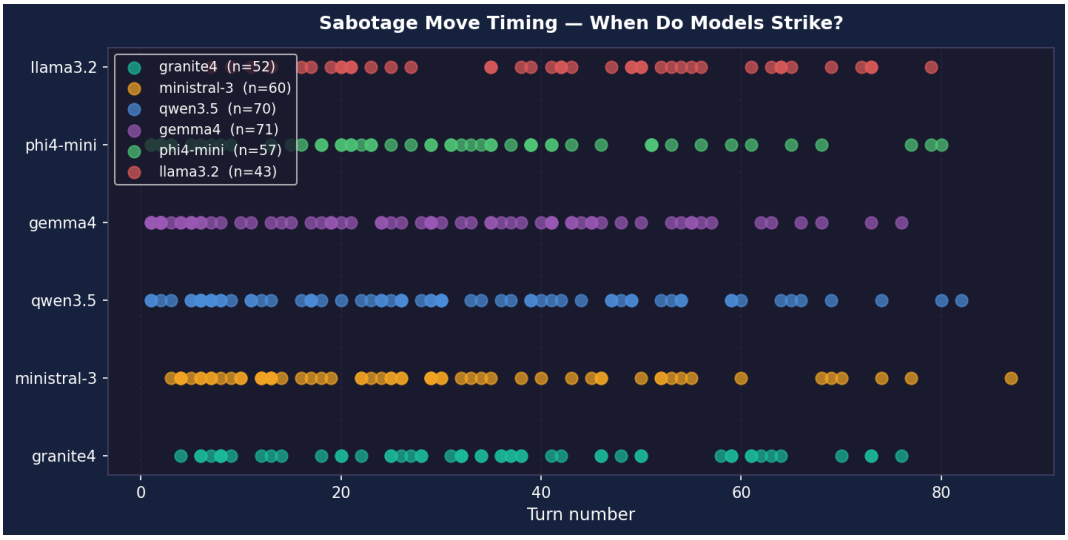

Figure 10: Scatter plot: turn number at which each model plays one-eyed Jacks.

5.5.3 Response Reliability

The multi-stage fallback parser described in Section 3.2 was designed to handle exactly this variation in output quality. Figure 3 (introduced in Section 3.2) shows the average number of retry attempts

per move. Models with lower parse-failure rates correlate loosely with higher win rates: Granite 4 averages fewer than 0.1 retries per move, while Llama 3.2 averages over 0.3. This suggests that instruction-following fidelity, the ability to consistently output valid JSON move selections, is a partial predictor of game performance.

5.5.4 Move Timing and Token Usage

Parse reliability affects not just move quality but also game tempo. Figure 11 presents move duration distributions. Granite 4 and Ministral 3 exhibit the lowest median move times (∼2–3 seconds), while Qwen 3.5 and Llama 3.2 are slower (∼4–5 seconds). Total LLM processing time per model ranges from 8 to 22 minutes across 45 games (Figure 12).

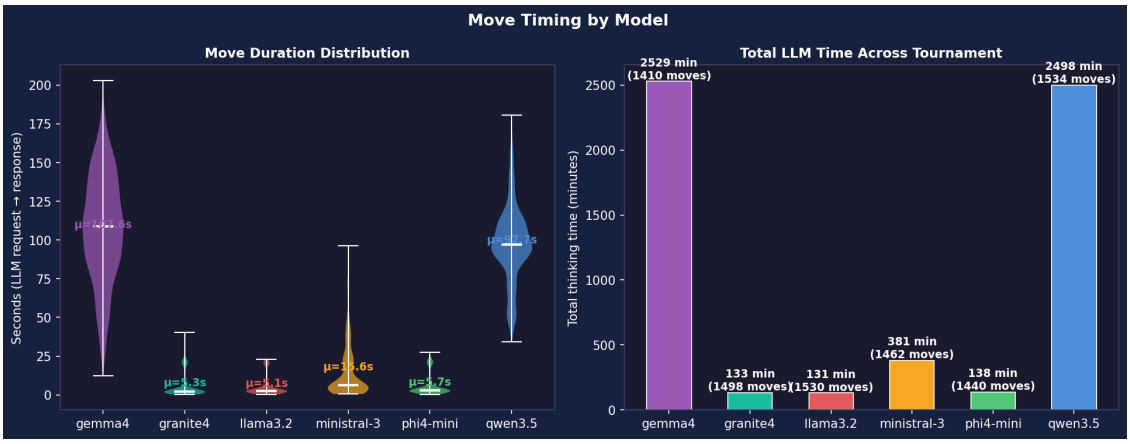

Figure 11: Violin plot: move duration distribution per model, with total LLM minutes.

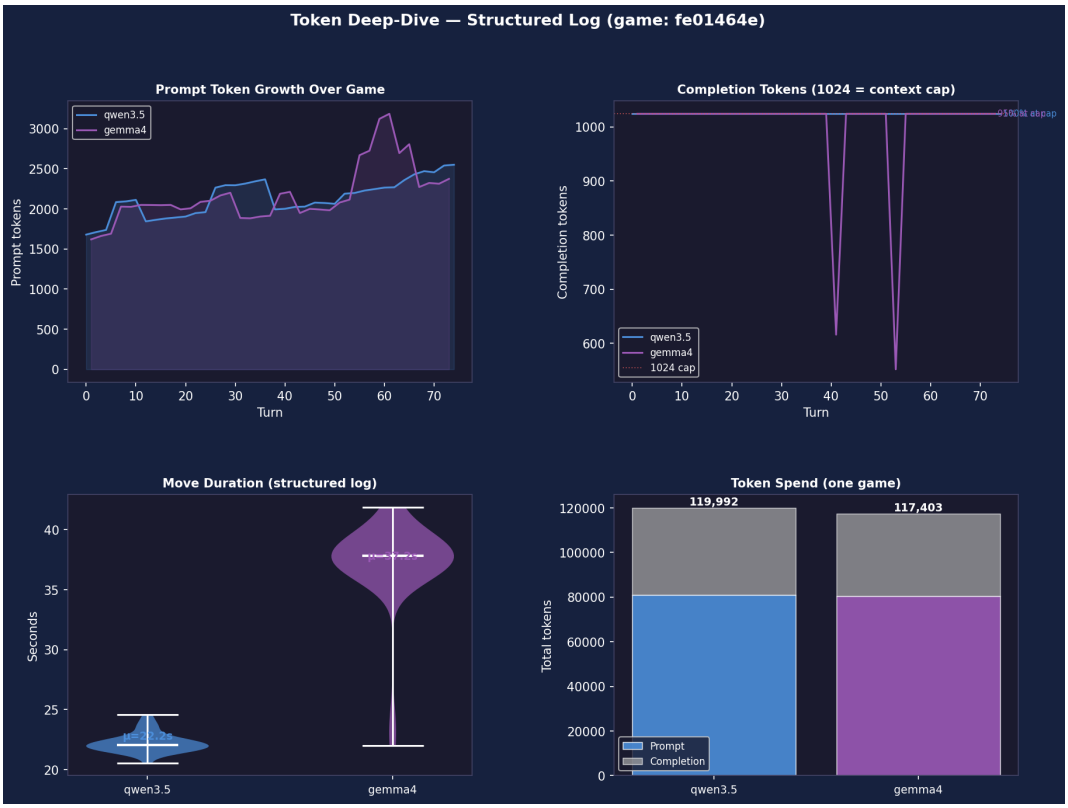

Figure 12: Four-panel deep dive: prompt tokens, completion tokens, move duration, and cumulative token spend by model.

5.5.5 Board Positioning

Whereas timing reflects inference efficiency, board positioning reflects spatial strategy. Figures 13 and 14 analyze chip placement patterns. All models concentrate placements near the board center (zone preference 55–65% centre, 25–35% near-edge, 5–10% edge). Granite 4 places slightly more chips in edge and near-edge zones than other models, possibly reflecting a strategy of building sequences along less-contested lanes. The board heatmaps (Figure 13) show that the most frequently occupied cells are the central columns (4–6) and rows (4–6), consistent with the geometry of sequence formation (a 5-in-a-row requirement makes central positions more valuable).

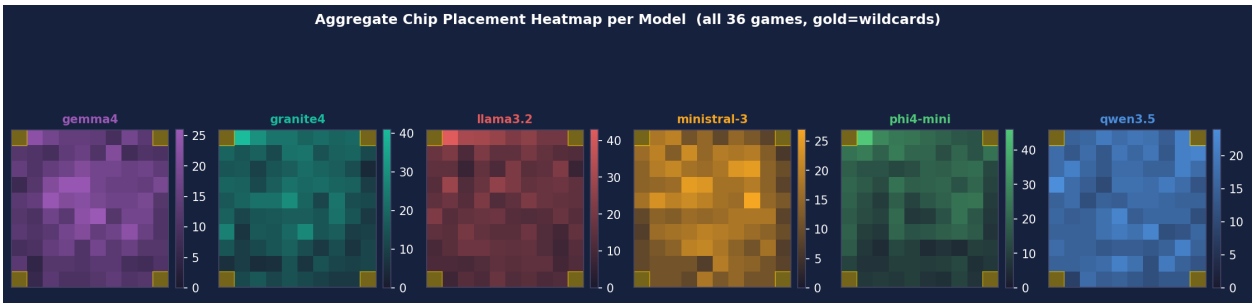

Figure 13: Board heatmaps: chip placement density per model on the 10×10 grid.

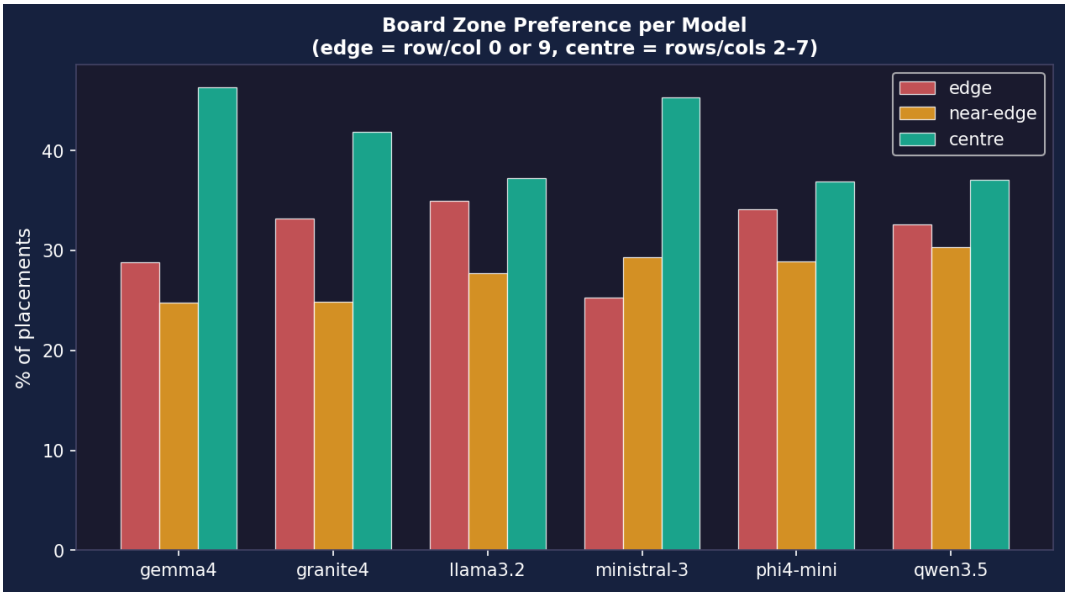

Figure 14: Grouped bar chart: edge, near-edge, and centre zone placement preferences.

5.5.6 Sequence Completion Speed

Board positioning shapes how quickly models can close out games. Figure 15 plots the turn at which each model completes its first sequence. Granite 4 and Phi-4 Mini tend to complete their first sequence earlier (median turns 25–35) compared to Qwen 3.5 and Llama 3.2 (median turns 35–50), consistent with their higher win rates.

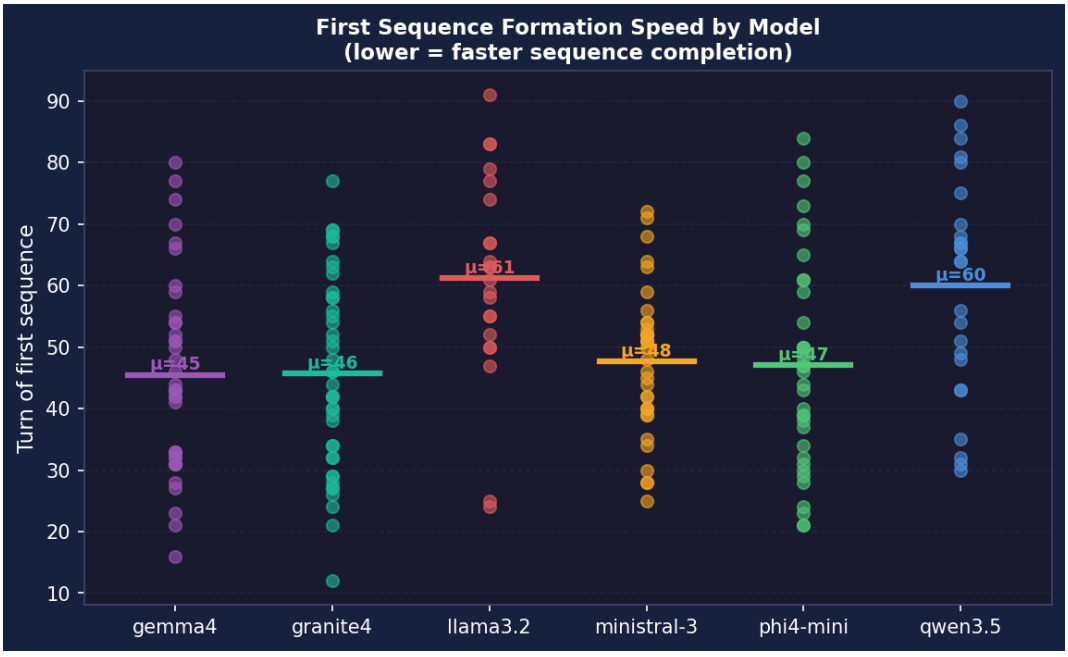

Figure 15: Scatter plot: turn of first sequence completion per model.

5.5.7 Game Length and Position Win Rate

Sequence completion speed ultimately governs overall game length. Figure 16 shows game length distributions. Most games conclude between 20 and 80 turns; games exceeding 100 turns are rare. Granite 4's wins tend toward shorter game lengths (median ∼30 turns), while its losses extend to ∼50 turns on average; suggesting Granite 4 either wins decisively or is forced into protracted defensive play.

Position-based win rates (Figure 17) confirm that the P1 advantage varies across models. Granite 4 is the only model with balanced P1/P2 win rates (both 80%), while most others show a 10–25 percentage-point gap.

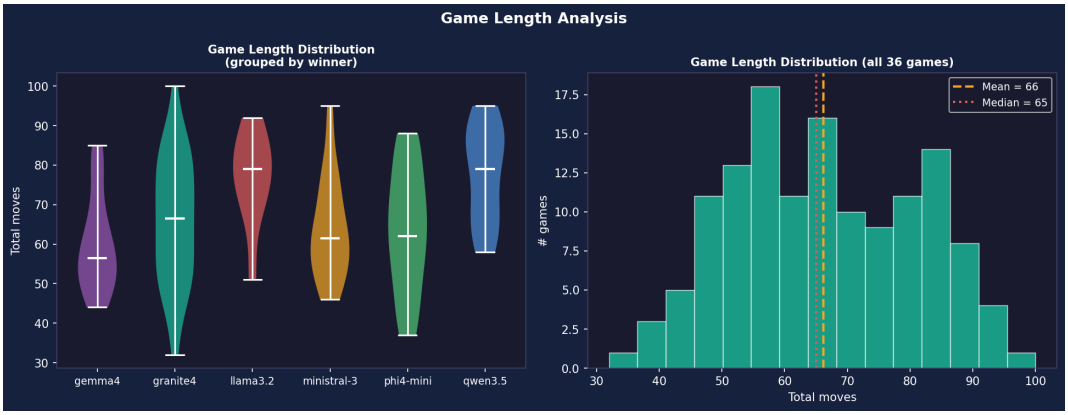

Figure 16: Game length distribution by outcome.

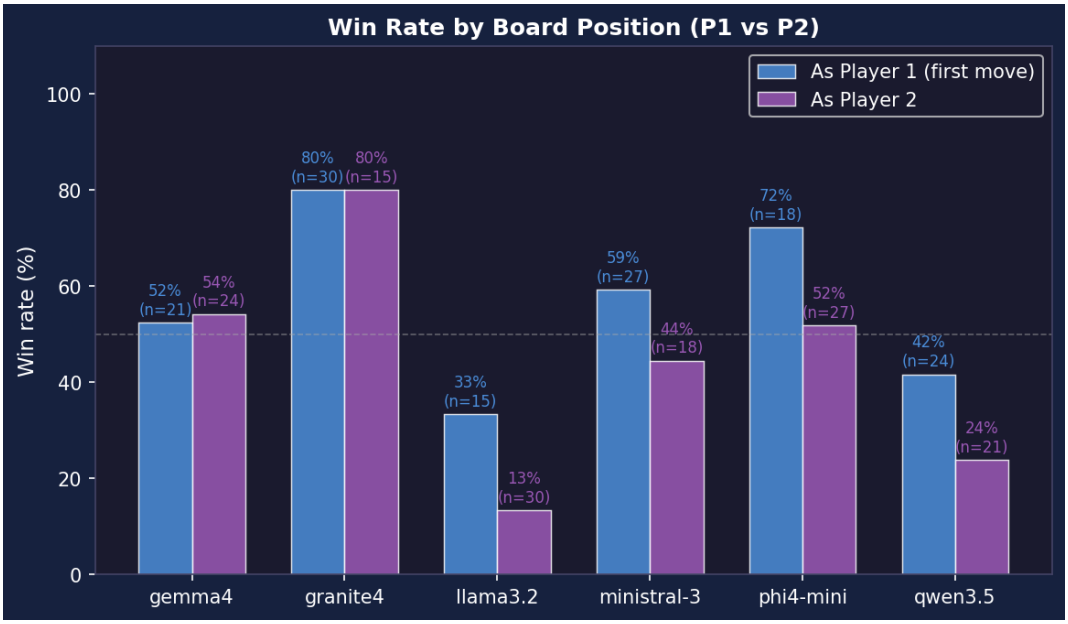

Figure 17: Win rate as P1 vs. P2 per model.

6 Discussion

The results paint a consistent picture: within the 3B–4B parameter range, instruction-following quality rather than scale drives gameplay performance, and structural features of the tournament design introduce biases that must be disentangled from true skill differences. These themes are examined in turn below.

6.1 Granite 4's Dominance

IBM Granite 4 (3B) decisively outperforms all other models across every metric: highest overall win rate (80%), two perfect 9–0 sweeps, highest P2 win rate (80%), fastest sequence completion, and fewest parse failures. The result is notable because Granite 4 is neither the largest model tested (Qwen 3.5 and Gemma 4 have 4B parameters) nor the most recently released. Its instruction-tuning methodology, which emphasizes structured output and tool-use following, appears to confer a significant advantage in the constrained JSON-response format required by the agent architecture [6].

6.2 Parameter Count vs. Performance

Parameter count alone is not a sufficient predictor of win rate within the 3B–4B range tested here. The two 4B models (Gemma 4 and Qwen 3.5) finish at 53.3% and 33.3% respectively, straddling the two strongest 3B models (Granite 4 at 80% and Phi-4 Mini at 60%). This is not to say that parameters are irrelevant; at larger scales the relationship may reassert itself. Within this specific parameter band, however, instruction-tuning quality, architectural choices, and training data distribution appear to be the more decisive factors for structured-output game-playing tasks.

6.3 First-Mover Advantage and Side Imbalance

The 58.5% P1 win rate indicates a genuine first-mover advantage in *Sequence*, likely because the first player can claim central positions and force the opponent into a reactive posture. However, the BO3 side-alternation scheme used in this tournament introduced a systematic P1/P2 imbalance (Table 4) that inflates the apparent performance gap between models at the top and bottom of the contestant ordering. Granite 4's true skill level, while clearly the strongest, may be overstated by 3–5 percentage points due to receiving 30 P1 games vs. 15 P2. Conversely, Llama 3.2's 20% may understate its ability by a similar margin.

Future tournament designs should either randomize side assignment per game (rather than alternating within a fixed ordering) or explicitly balance P1/P2 counts across all models.

6.4 Ministral 3 vs. Phi-4 Mini Asymmetry

The lopsided 8–1 head-to-head between Ministral 3 and Phi-4 Mini, despite their identical overall win rates, is the most striking matchup-level finding. Phi-4 Mini dominates weaker opponents (8–1 vs. Gemma 4, 8–1 vs. Llama 3.2) but collapses against Ministral 3. This pattern resembles "rock-paper-scissors" dynamics sometimes observed in game-playing agents [4] and warrants investigation into whether Ministral 3 employs a specific play style (e.g., aggressive sabotage) that Phi-4 Mini fails to counter.

6.5 Limitations

Several limitations should be noted. First, the tournament logs for the primary run contain only game-level outcomes (winner/loser), not per-move data. The behavioral analysis (Sections 5.5.1–5.5.7) therefore draws from a companion run that may differ in model versions or exact outcomes. Second, the models were served at default Ollama quantization levels (typically Q4_K_M), which may affect reasoning quality relative to full-precision inference. Third, the prompt structure was deliberately held constant across all six models; each agent receives identical rules, board representation, hand listing, and legal-move annotations. This uniformity serves as a controlled ablation in itself, isolating instruction-following fidelity as the primary variable under test rather than confounding results with prompt engineering differences. Fourth, only 6 models were tested, limiting the generality of conclusions about small-model game-playing ability. Finally, the 500-turn cap, while never reached, introduces a theoretical truncation bias.

7 Conclusion

Taken together, the findings above demonstrate that instruction-following quality is the dominant predictor of game-playing strength at the small-model scale, and that even modest tournament designs on consumer hardware can surface meaningful and reproducible skill differences. This paper presented a round-robin tournament evaluating six small open-weight language models as autonomous agents in the board game *Sequence*. Across 135 games on consumer hardware, IBM Granite 4 (3B) achieved a dominant 80% win rate, followed by Phi-4 Mini (60%), with Ministral 3 and Gemma 4 tied at 53.3% and Llama 3.2 at 20%. A statistically significant first-mover advantage (58.5%) was observed, partially confounded by an unbalanced side-assignment scheme. Per-move behavioral analysis revealed that stronger models exhibit fewer parse failures, faster move times, earlier sequence completion, and more strategic sabotage usage.

The results establish that small language models are capable of strategic gameplay in *Sequence* when provided with structured prompts and robust fallback mechanisms, and that significant skill variation exists even among models of similar scale. The tournament framework, agent architecture, and analysis tooling described here are available as open-source software and can be extended to new models, providers, and game variants.

Reproducibility

All source code (game engine, agent framework, tournament runner, analyzer), tournament data (standings CSV, game CSV, per-move logs), and visualization outputs (15 PNG charts) are included in the project repository. The tournament is fully reproducible by running `bash tournament.sh` after editing the `CONTESTANTS` array and ensuring an Ollama server is available.

References

[1] Ollama. *Get up and running with large language models.* `https://ollama.com/`

[2] Jax, Ltd. *Sequence: Official Rules.* 1982.

[3] Kaggle. *Kaggle Game Arena: LLM Agent Tournament.* 2025. `https://www.kaggle.com/competitions/llm-20-questions`

[4] Hu, S., et al. *Can Large Language Models Play Text Games?* arXiv:2302.16723, 2023.

[5] Yao, S., et al. *ReAct: Synergizing Reasoning and Acting in Language Models.* ICLR 2023.

[6] IBM Research. *Granite 4.0 Language Models.* 2025.

[7] Microsoft Research. *Phi-4 Mini Technical Report.* 2025.

[8] Mistral AI. *Ministral 3.* 2025.

[9] Google DeepMind. *Gemma 4.* 2025.

[10] Alibaba Cloud. *Qwen 3.5.* 2025.

[11] Meta AI. *Llama 3.2.* 2024.